Kế hoạch riêng cho Tiến
Cognitive / LLM Lead

1. Vai trò chính

Tiến phụ trách phần **agent logic**, **prompt**, **reasoning**, **SectorAgent**, **AllocatorAgent**, **Debate** và **Memory Sharing logic**.

Nói ngắn gọn:

Vai trò tổng quát

Tiến biến FINMEM gốc thành các agent có thể **tranh luận** và **chia sẻ ký ức**.

- Tiến **không quản FAISS ở tầng hạ tầng**, nhưng dùng API FAISS của Dạt.
- Tiến **không train GAT/Fusion**, nhưng dùng score của Hiệp để ra quyết định.

2. Folder Tiến phụ trách

Các folder / file chính

```
puppy/swarm/
puppy/debate/
puppy/memory_ext/memory_sharing.py
puppy/fusion/text_encoder.py
puppy/prompts.py
puppy/reflection.py
docs/llm_agent_design.md
```

Lưu ý:

- `reflection.py` và `prompts.py` là file lõi của FINMEM gốc.
- Hiện `reflection.py` đang ép output LLM bằng Guardrails và phân biệt train/test.
- `prompts.py` chứa prompt train/test, memory id description, sentiment explanation và momentum explanation.
- Vì vậy Tiến không nên sửa mạnh ngay, nên tạo prompt/debate file mới trước.

Phase T1 — Bọc LLMAgent thành SectorAgent

Thời gian: Tuần 1–2
Mục tiêu: Mỗi cổ phiếu/ngành là một agent riêng.

Code gốc `LLMAgent` đang có:

- `BrainDB`
- `Portfolio`
- `character_string`
- `trading_symbol`
- `top_k`
- `reflection_result_series_dict`
- hàm `step()`

File cần tạo

```
puppy/swarm/sector_agent.py
puppy/swarm/protocols.py
```

Việc cần làm

1. Tạo class `SectorAgent`.
2. Bên trong `SectorAgent` chứa một `LLMAgent` gốc.
3. `SectorAgent.step()` gọi `LLMAgent.step()`.
4. `SectorAgent.get_signal()` lấy `decision`, `reason`, `memory_id` từ reflection result.
5. `SectorAgent` có `external_context_buffer` để sau này nhận memory từ agent khác.

Output cuối phase

- TSLA `SectorAgent` chạy được.
- NVDA `SectorAgent` chạy được.
- AAPL `SectorAgent` chạy được.
- Mỗi agent output `AgentSignal` chuẩn.

Phase T2 — AllocatorAgent bản đầu tiên

Thời gian: Tuần 3–4
Mục tiêu: Có agent trung tâm gom tín hiệu.

File cần tạo

`puppy/swarm/allocator_agent.py`

Việc cần làm

1. Nhận list `AgentSignal` từ các `SectorAgent`.
2. Bản MVP dùng rule đơn giản:
   - nhiều `buy` hơn `sell` ⇒ tăng allocation;
   - nhiều `sell` hơn `buy` ⇒ giảm allocation;
   - `hold` ⇒ giữ nguyên.
3. Lưu reason:

   Allocator phải giải thích tại sao phân bổ vốn như vậy.

Output cuối phase

- Allocator gom được tín hiệu từ nhiều `SectorAgent`.
- Có `AllocationDecision`.
- Chưa cần debate ở phase này.

Phase T3 — Text Stream Encoder

Thời gian: Tuần 5–7
Mục tiêu: Tạo `h_text` cho Hiệp dùng trong Fusion.

File cần tạo

`puppy/fusion/text_encoder.py`

Việc cần làm

1. Lấy text từ:
    - latest decision reason;
    - top shallow memory;
    - top mid memory;
    - top long memory;
    - reflection memory.
2. Gộp thành agent context.
3. Gọi embedding để tạo `h_text`.
4. Trả về:
    - `symbol`
    - `date`
    - `h_text`
    - `raw_context`

Hiện repo đã có `embedding.py` dùng OpenAI embedding và trả vector dạng `numpy float32`.

Output cuối phase

- Mỗi `SectorAgent` có text feature.
- Hiệp có thể dùng `h_text` để train Fusion.

Phase T4 — Dynamic Memory Sharing logic

Thời gian: Tuần 8–10
Mục tiêu: Khi Hiệp báo agent A ảnh hưởng agent B, Tiến tạo logic chia sẻ ký ức.

Draft paper mô tả **Dynamic Memory Sharing**: GNN đóng vai trò router. Nếu ảnh hưởng từ agent A sang agent B vượt ngưỡng, hệ thống chia sẻ context text sang **Working Memory** của agent B.

File cần tạo

```
puppy/memory_ext/memory_sharing.py
puppy/swarm/sector_agent.py
```

Việc cần làm

1. Nhận trigger từ `risk_router` của Hiệp:

   ```
   source_agent = TSLA
   target_agent = NVDA
   alpha_ij = 0.82
   ```

2. Gọi API search memory của Đạt:

   ```
   search_memory(agent_id="TSLA", layer="short", top_k=3)
   ```

3. Tóm tắt memory được chia sẻ:

   > TSLA đang chịu ảnh hưởng bởi tin X, có thể ảnh hưởng chuỗi cung ứng bán dẫn...

4. Inject sang target agent:

   ```
   target_agent.inject_external_context(shared_context)
   ```

5. Khi target agent reflection:

   > `external_context` được đưa vào prompt hoặc reflection memory.

Output cuối phase

- Agent B nhận được context từ Agent A.
- Reason của Agent B có thể nhắc tới shared memory.
- Có log memory sharing.

Phase T5 — Debate Mechanism

Thời gian: Tuần 11–15
Mục tiêu: Khi text và graph xung đột, Allocator chất vấn SectorAgent.

Draft paper định nghĩa **Filter-then-Debate**: confidence zone thì xử lý nhanh, còn grey zone hoặc xung đột thì bật tranh biện.

File cần tạo

```
puppy/debate/prompts.py
puppy/debate/debate_loop.py
puppy/debate/judge.py
puppy/debate/conflict_detector.py
```

Việc cần làm

1. **Conflict Detector**
   Nếu fusion score nằm trong khoảng:

   $$0.3 \leq \text{fusion_score} \leq 0.7$$

   hoặc:

   $$\text{text_signal} \neq \text{graph_signal}$$

   thì bật debate.

2. **Interrogation**
   Allocator hỏi:

   > Graph báo rủi ro cao, tại sao SectorAgent vẫn BUY?

3. **Justification**
   SectorAgent truy xuất Deep Memory / Reflection để bảo vệ quan điểm.

4. **Judge**
   Allocator quyết định:

   - giữ nguyên;
   - giảm allocation;
   - override thành `hold` / `sell`.

Output cuối phase

- Debate chỉ bật khi cần.
- Có question / answer / verdict.
- Có output override decision.

Phase T6 — Explainability cho paper / UI

Thời gian: Tuần 16–20

File cần tạo

```
puppy/evaluation/explainability_report.py
docs/debate_examples.md
docs/memory_sharing_examples.md
```

Việc cần làm

1. Chọn vài case study:
    - Agent BUY nhưng graph risk cao;
    - Agent SELL vì memory sharing từ agent khác;
    - Allocator override decision sau debate.
2. Viết explanation:
    - memory id nào được dùng;
    - lý do agent quyết định;
    - vì sao allocator giữ hoặc override.
3. Chuẩn bị nội dung cho báo cáo / paper.